

IMPLEMENTARE UN PL

→ Costruire la macchina astratta per il linguaggio
(insieme di algoritmi e strutture dati che interagiscono potenzialmente con una memoria)

$\forall M_L \exists$ un unico L che
è il linguaggio riconosciuto ed eseguito da M_L
(linguaggio macchina di M_L)

$\forall L$ PL possono esistere infinite macchine astratte per L

Traduzione implicita interpretazione → "traduzione" istruzione per istruzione
in effetto sullo stato

Traduzione esplicita compilazione → "traduzione" effettiva del programma in
un PL già implementato

Interprete:

I^{L_0L} è un programma scritto in L_0

$I^{L_0L} : \text{Prog}^L \times D \rightarrow D$ D insieme di dati

t.c. $I^{L_0L}(P, d) = P(d)$ macchina universale

Struttura dell'interprete → ciclo stile "fetch-execute"

[0:14:00]

Compilatore

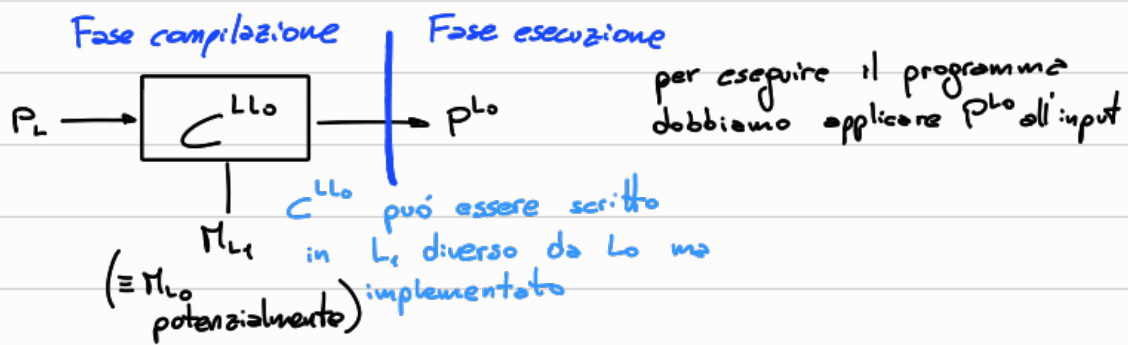
Linguaggio da implementare → L
già implementato → L_0
Definizione: compilatore da L a L_0 è un programma

$C^{L_0L} : \text{Prog}^L \rightarrow \text{Prog}^{L_0}$

$\forall P^L \in \text{Prog}^L . C^{L_0L}(P^L) = P^{L_0}$ t.c. $\forall d \in D . P^{L_0}(d) = P^L(d)$ // no dati

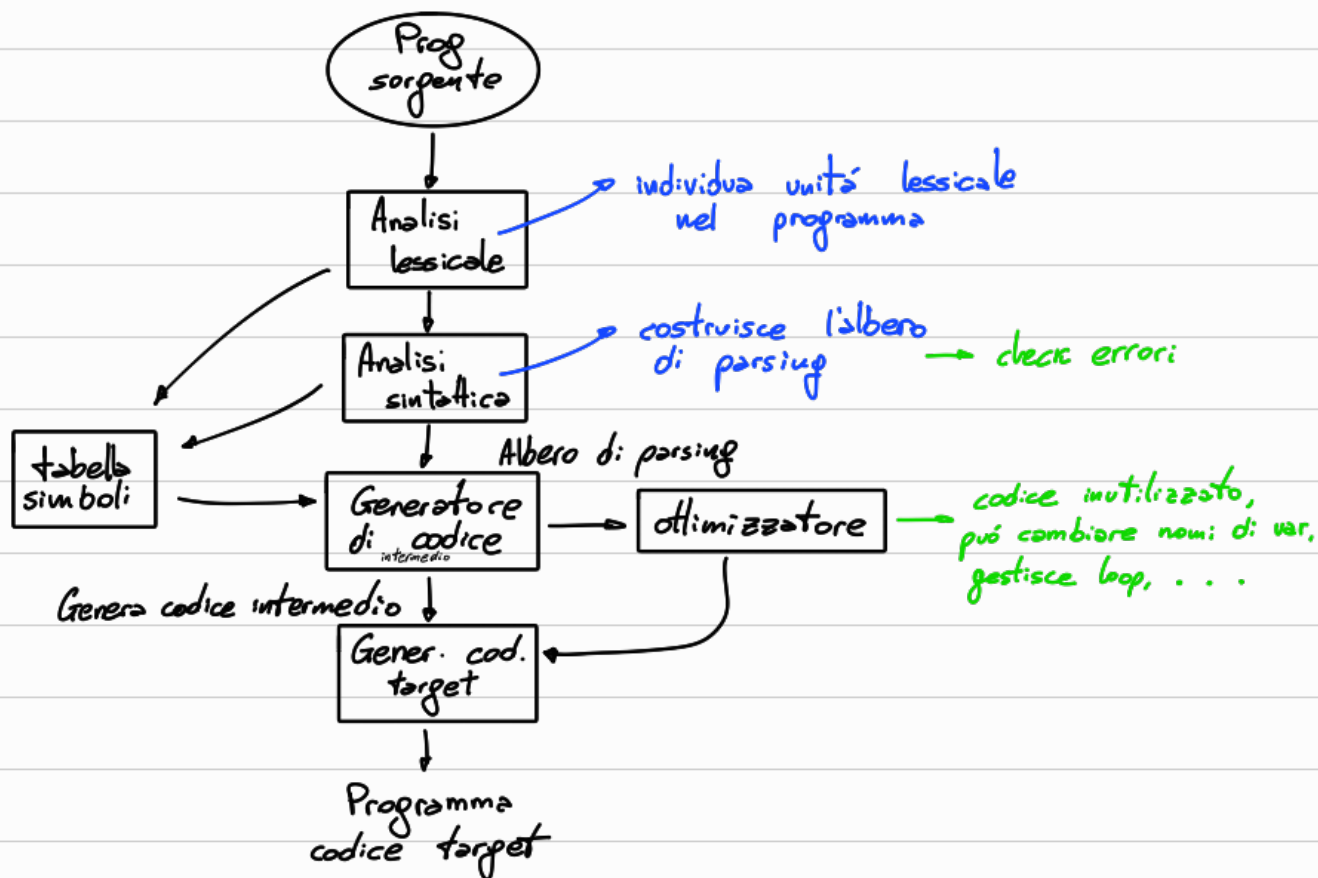
non decidibile

non posso dimostrare se
il compilatore va bene
(infinit. dati, potenzialmente divergente)



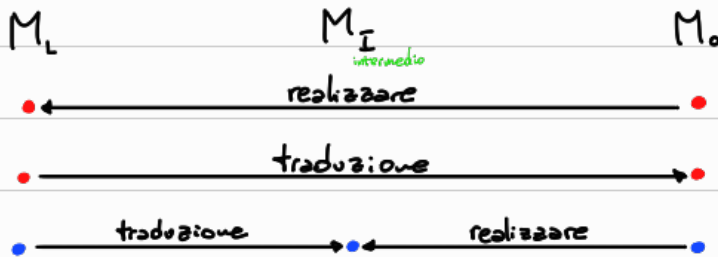
→ traduce in un codice diverso
 => separa la traduzione dall'esecuzione

Struttura compilatore



L ling. da impl.

L_0 PL eseguibile



interprete

compilatore

ibridi (java)

[0:42:00]

Specializzatore

Uno specializzatore per un PL L è un programma che realizza la funzione $\text{spec}_L : \text{Prog}^L \times D \rightarrow \text{Prog}^L$

$$\forall P^L \in \text{Prog}^L, \forall d \in D. \text{spec}_L(P^L, d) = P^L_d \quad \text{t.c.} \quad P^L(d, y) = P^L_d(y)$$

ma Lo specializzatore fornisce un legame tra interprete e compilatore

$\Rightarrow$ Specializzando un interprete (scritto in L_0) sul programma (scritto in L) otteniamo un compilatore da L a L_0

$I(P^L, d)$
↑
scritto in L_0

$\text{Spec}(I, P^L)$ è un programma scritto in L_0 che integra nel codice l'esecuzione di P^L

[0:56:07]

Descrivere : linguaggi di programmazione

Sintassi = regole di formazione

(determiniamo quali sono le frasi ben formate nel linguaggio)

modo con cui rappresentato un valore

→ descrive relazioni tra simboli

Semantica = Attribuzione del significato ai simboli

(determina il significato delle frasi ben formate nel linguaggio)

valore

→ descrive relazioni tra simboli e significato

Pragmatica = Regole di buon utilizzo del linguaggio

suggerimenti:
//non usare eval() per ragioni di sicurezza

Implementazione = Esistono strumenti che eseguono frasi ben formate del linguaggio

SINTASSI

Parola $\rightarrow$ stringa su un alfabeto Σ

Frase $\rightarrow$ σ una sequenza di parole (ben formato se rispetta le regole della sintassi)

Linguaggio $\rightarrow$ insieme di frasi ben formate

Ling. $\{a^n b^n \mid n \in \mathbb{N}\} \rightarrow$ PL insieme di programmi

Alf $\Sigma = \{a, b\} \rightarrow \{if, while, then, else\}$

stringhe $\varepsilon, ab, a^2 b^2, \dots \rightarrow$ Programmi (seq. di parole)

Lessema $\rightarrow$ parola con significato (costituita da soli terminali nell'alfabeto) parole = simboli primitivi

if exp then com else com

Token $\rightarrow$ categorie sintattiche (simboli non terminali)

if $x > 0$ then $x = x - 1$ else $x = x + 1$

Strumenti di definizione della sintassi

Riconoscitore

Strumento per il riconoscimento di stringhe appartenenti al linguaggio

Esempio minimo: automi a stati finiti

$\hookrightarrow$ Analisi lessicale per riconoscimento di lessemi

Generatore

Strumento per la generazione di stringhe

$\hookrightarrow$ compilazioni per riconoscere programmi scritti in PL (parsing)

Una grammatica CF $\rightarrow$ per PL $G = \langle V, T, P, S \rangle$

• V insieme finito di simboli non terminali

• T insieme finito di simboli terminali categorie sintattiche vocabolario ($V \cap T = \emptyset$)

• P insieme finito di produzione regole di formulazione

$A \rightarrow \alpha \quad A \in V \quad \alpha \in (T \cup V)^*$

• $S \in V$ simbolo iniziale categoria della frase (programma)

$\hookrightarrow$ Notazione BNF spesso usato per i PL

Es: $\rightarrow$ $\leadsto$ $::=$

Esempio: grammatica per espressioni booleane

$G = \langle \{Exp\}, \{or, and, not, (,), 0, 1\}, P, Exp \rangle$

$\hookrightarrow Exp \rightarrow 0$

$Exp \rightarrow 1$

$Exp \rightarrow (Exp \text{ or } Exp)$

$Exp \rightarrow (not \ Exp)$

$Exp \rightarrow 0 \mid 1 \mid (Exp \text{ or } Exp) \mid (Exp \text{ and } Exp) \mid (not \ Exp) \mid$